LABORATORY OBSERVATION

Course:	Full Stack Web Development
Course Code:	23CM4121
Experiment No.:	4
Student Name:	P.Dinesh Kumar
Roll No.:	A24126552101
Date:	30-08-2026

---

1. TITLE

Interactive To-Do List Application Using JavaScript DOM Event Listeners

2. AIM

To implement a dynamic To-Do List web application featuring task creation, event listeners, complete toggle state, task deletion, and empty state management.

3. OBJECTIVE

The objectives of this experiment are:

- To handle user click events using addEventListener.
- To dynamically create HTML elements (document.createElement).
- To toggle CSS classes (.completed) for line-through strikeout effect.
- To remove elements from DOM and update empty list messages.

4. THEORY

Event listeners capture interactive user actions. Dynamic element creation and classList toggling allow real-time list item manipulation in single-page applications.

Application Flow:

```
Input Task Text -> Add Event Listener -> createElement("div") -> Append Buttons -> Toggle Complete / Remove Task
```

5. ALGORITHM / PROCEDURE

1. Create task4.html with input field, add button, and task list container.
2. Attach click event listener to add button.
3. Validate input text and hide empty list message.
4. Create task container, text span, Complete button, and Delete button.
5. Attach toggle listener for line-through style and removal listener for delete button.

6. PROGRAM

task4.html

```
<!DOCTYPE html>
<html>

<head>

    <title>My To-Do List</title>

    <style>

        body {
            font-family: Arial;
            background-color: #eaf4ff;
            text-align: center;
            margin: 40px;
        }

        h1 {
            color: #174a7e;
        }

        input {
            width: 350px;
            padding: 10px;
            border: 1px solid gray;
            border-radius: 5px;
        }

        button {
            padding: 10px 15px;
            margin: 5px;
            border: none;
            border-radius: 5px;
            background-color: #174a7e;
            color: white;
        }

        #taskList {
            width: 500px;
            margin: 25px auto;
        }

        .task {
            background-color: white;
            border: 1px solid #aaa;
            padding: 10px;
            margin: 10px;
            border-radius: 5px;
        }

        .completed {
            text-decoration: line-through;
            color: gray;
        }

        .delete {
            background-color: #d9534f;
```

```
        }

        #emptyMessage {
            color: gray;
        }

    </style>

</head>

<body>

    <h1>My To-Do List</h1>

    <input type="text" id="taskInput" placeholder="Enter a task">

    <button id="addButton">Add Task</button>

    <div id="taskList">

        <p id="emptyMessage">No tasks available.</p>

    </div>

    <script>

        var taskInput = document.getElementById("taskInput");
        var addButton = document.getElementById("addButton");
        var taskList = document.getElementById("taskList");
        var emptyMessage = document.getElementById("emptyMessage");

        addButton.addEventListener("click", function() {

            var taskText = taskInput.value;

            if (taskText == "") {
                return;
            }

            emptyMessage.style.display = "none";

            var task = document.createElement("div");
            task.className = "task";

            var text = document.createElement("span");
            text.innerHTML = taskText;

            var completeButton = document.createElement("button");
            completeButton.innerHTML = "Complete";

            var deleteButton = document.createElement("button");
            deleteButton.innerHTML = "Delete";
            deleteButton.className = "delete";

            completeButton.addEventListener("click", function() {
                text.classList.toggle("completed");
            });

            deleteButton.addEventListener("click", function() {

                task.remove();
```

```
                if (taskList.children.length == 1) {
                    emptyMessage.style.display = "block";
                }

            });

            task.appendChild(text);
            task.appendChild(completeButton);
            task.appendChild(deleteButton);

            taskList.appendChild(task);

            taskInput.value = "";

        });

    </script>

</body>

</html>
```

7. CODE EXPLANATION

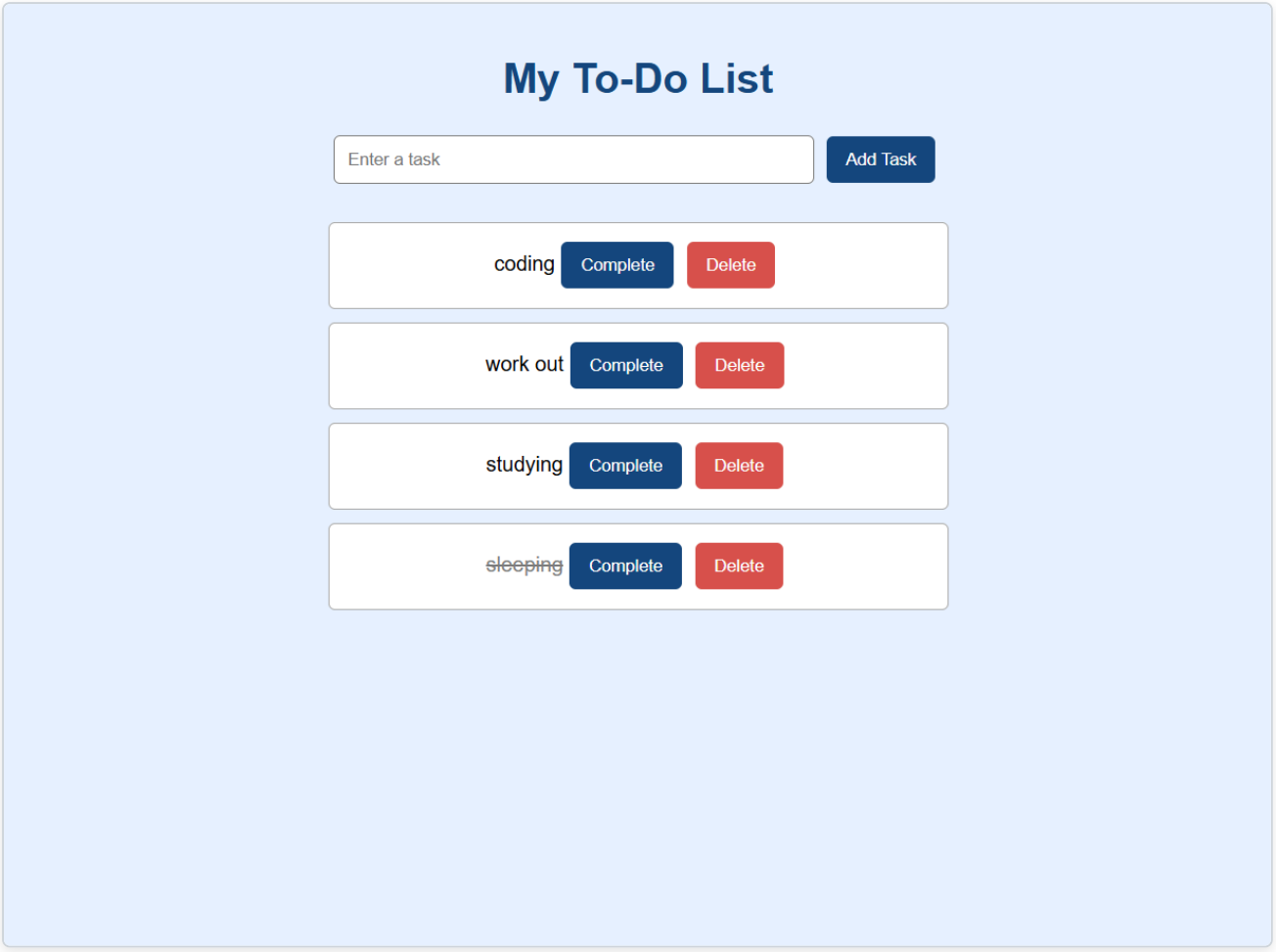
Code / Syntax	Explanation
`addEventListener("click", function)`	Attaches click event handler to button element.
`document.createElement("div")`	Creates new HTML div DOM element in memory.
`text.classList.toggle("completed")`	Toggles line-through text decoration style.
`task.remove()`	Removes task element directly from the DOM tree.

8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Add Task	Type text & click Add	New task div appended to task list container	**Pass**
TC-02	Complete Task	Click Complete button	Task text receives line-through style	**Pass**
TC-03	Delete Task	Click Delete button	Task removed from DOM & empty message displays if empty	**Pass**

9. OUTPUT



Output 1: Interactive To-Do List Application Using JavaScript DOM Event Listeners Rendered Webpage

10. RESULT

Thus, interactive to-do list application using javascript dom event listeners was successfully implemented and verified across all test cases.